

Advanced Programming 1
Assignment 3 – Project Design & Setup Milestone
Theme: Booking System
Group: SE-2425, Lenara Symbatkyzy, Nursalim Onalbayev.

Goal:

*Check whether the team understands what they are building and how it will be built.
This assignment evaluates planning and architecture, not full implementation.
Milestone 1 of the final project.*

1. Project Proposal

2. Architecture & Diagrams

3. Project Plan (Gantt)

4. Git Repository Setup

README.md

GoBook: Integrated Booking System (Milestone 1)

This repository contains the first milestone for the Advanced Programming 1 project. **GoBook** is a monolithic booking engine designed for performance and scalability using Go.

1. Project Proposal

Relevance & Target Users

GoBook addresses the need for a lightweight, high-concurrency booking solution for small to medium businesses. Unlike heavy enterprise software, GoBook offers a streamlined REST API for managing time-sensitive resources like co-working desks, medical appointments, or equipment rentals.

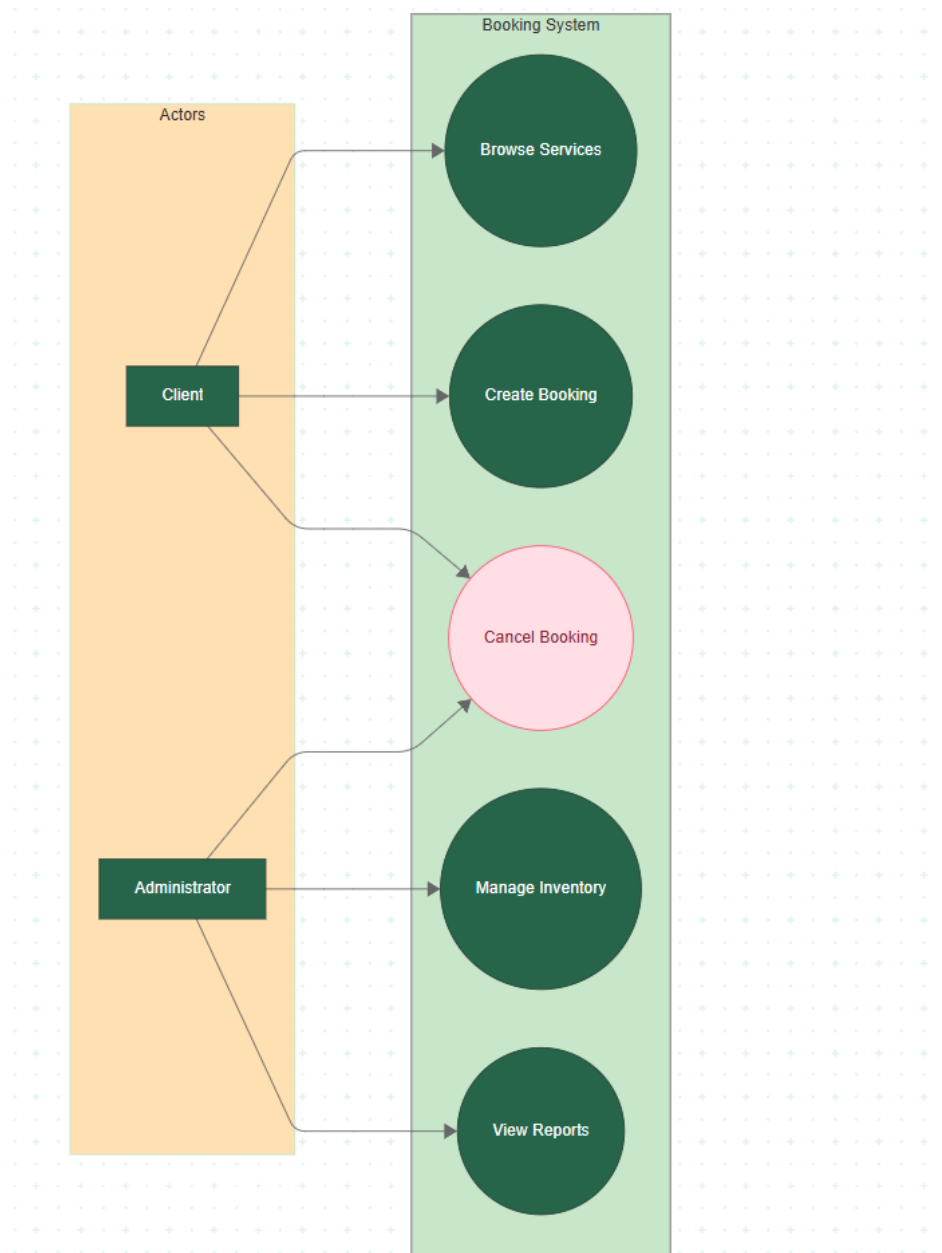
Planned Features

- **User Management:** JWT-based authentication and role-based access.
- **Resource Catalog:** CRUD operations for bookable items with custom pricing.
- **Conflict-Free Booking:** Logic to prevent overlapping schedules (double-booking).
- **Status Workflow:** Track bookings through Pending, Confirmed, and Completed states.

2. Architecture & Design

We utilize a **Modular Monolith** architecture. This ensures that while the application is a single binary, the internal logic is strictly separated into layers to allow for future migration to microservices if needed.

System Architecture Use-Case Diagram



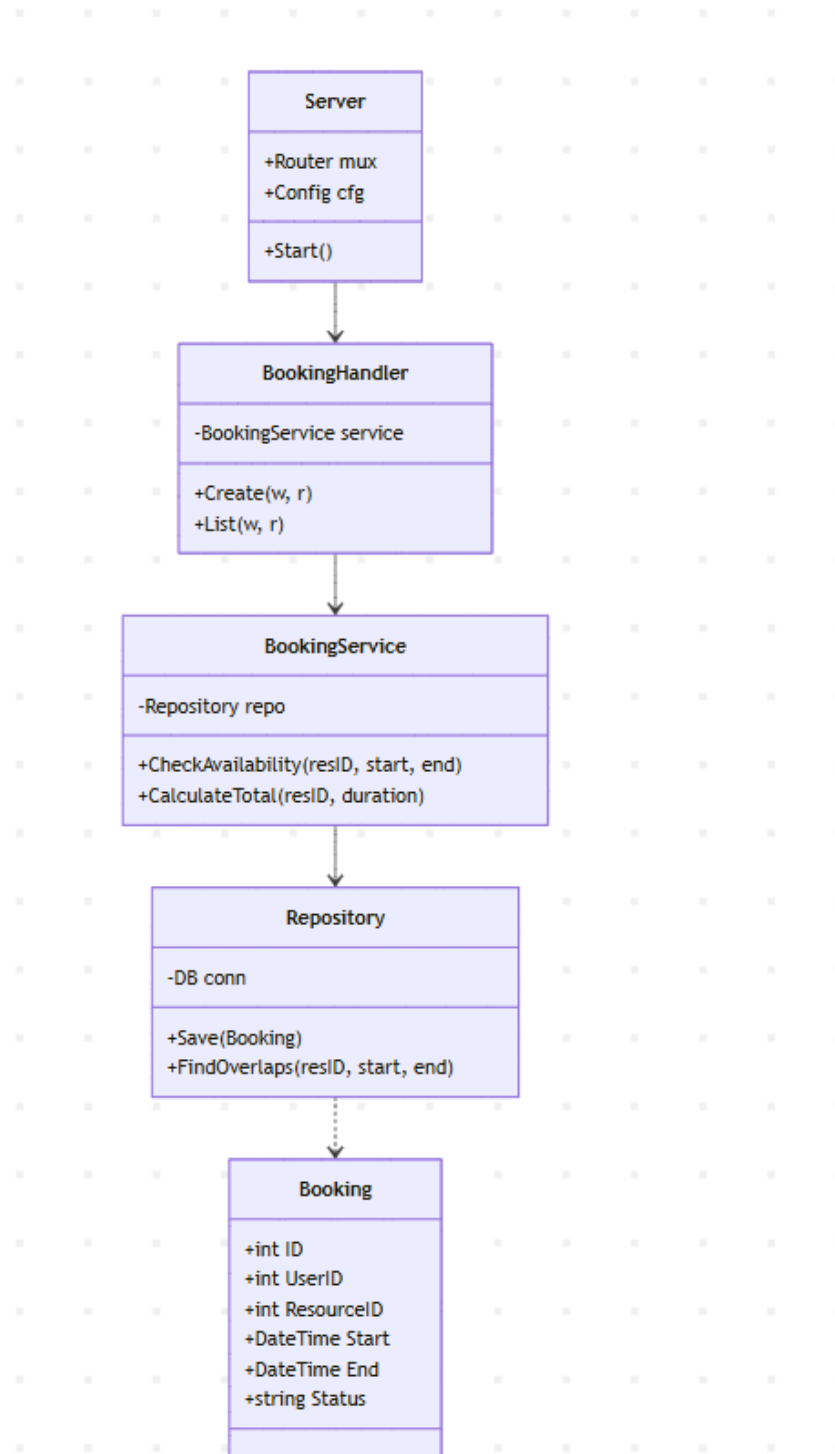
```
graph LR
    subgraph Users
        C[Client]
        A[Administrator]
    end

    subgraph "Booking System"
        UC1((Browse Resources))
        UC2((Make Reservation))
        UC3((Cancel Booking))
        UC4((Manage Inventory))
        UC5((Audit Logs))
    end

    C --> UC1
    C --> UC2
    C --> UC3
    A --> UC3
    A --> UC4
    A --> UC5
```

C --- UC1
C --- UC2
C --- UC3
A --- UC4
A --- UC5
A --- UC3

Detailed UML Class Diagram



```

classDiagram
    direction TB
    class Server {
        +Router mux
        +Config cfg
        +Start()
    }
    class BookingHandler {
        -BookingService service
        +Create(w, r)
        +List(w, r)
    }
    class BookingService {
        -Repository repo
        +CheckAvailability(resID, start, end)
        +CalculateTotal(resID, duration)
    }
    class Repository {
        -DB conn
        +Save(Booking)
        +FindOverlaps(resID, start, end)
    }
    class Booking {
        +int ID
        +int UserID
        +int ResourceID
        +DateTime Start
        +DateTime End
        +string Status
    }

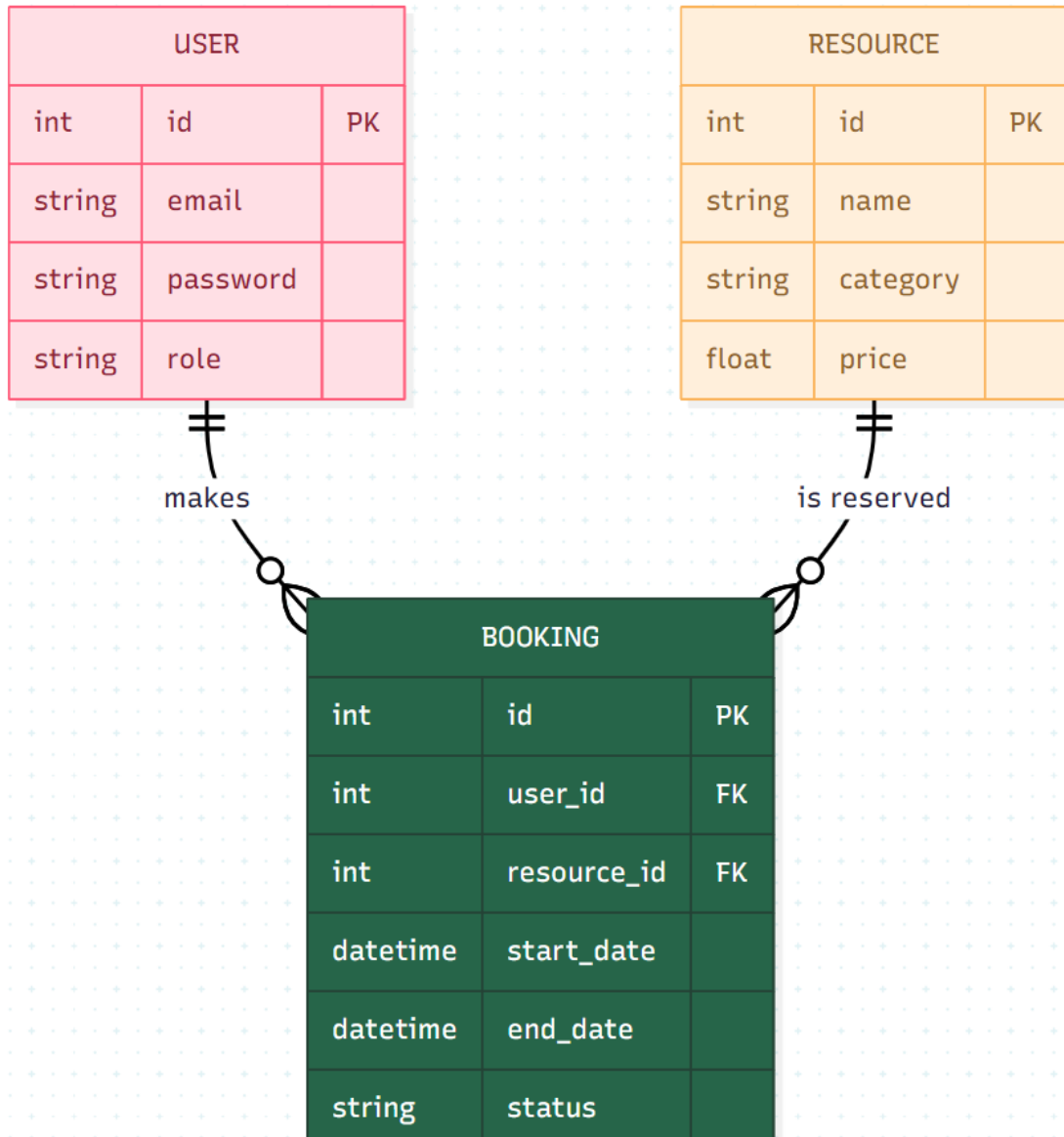
```

```

Server --> BookingHandler
BookingHandler --> BookingService
BookingService --> Repository
Repository ..> Booking

```

Entity Relationship Diagram (ERD)



ERDiagram

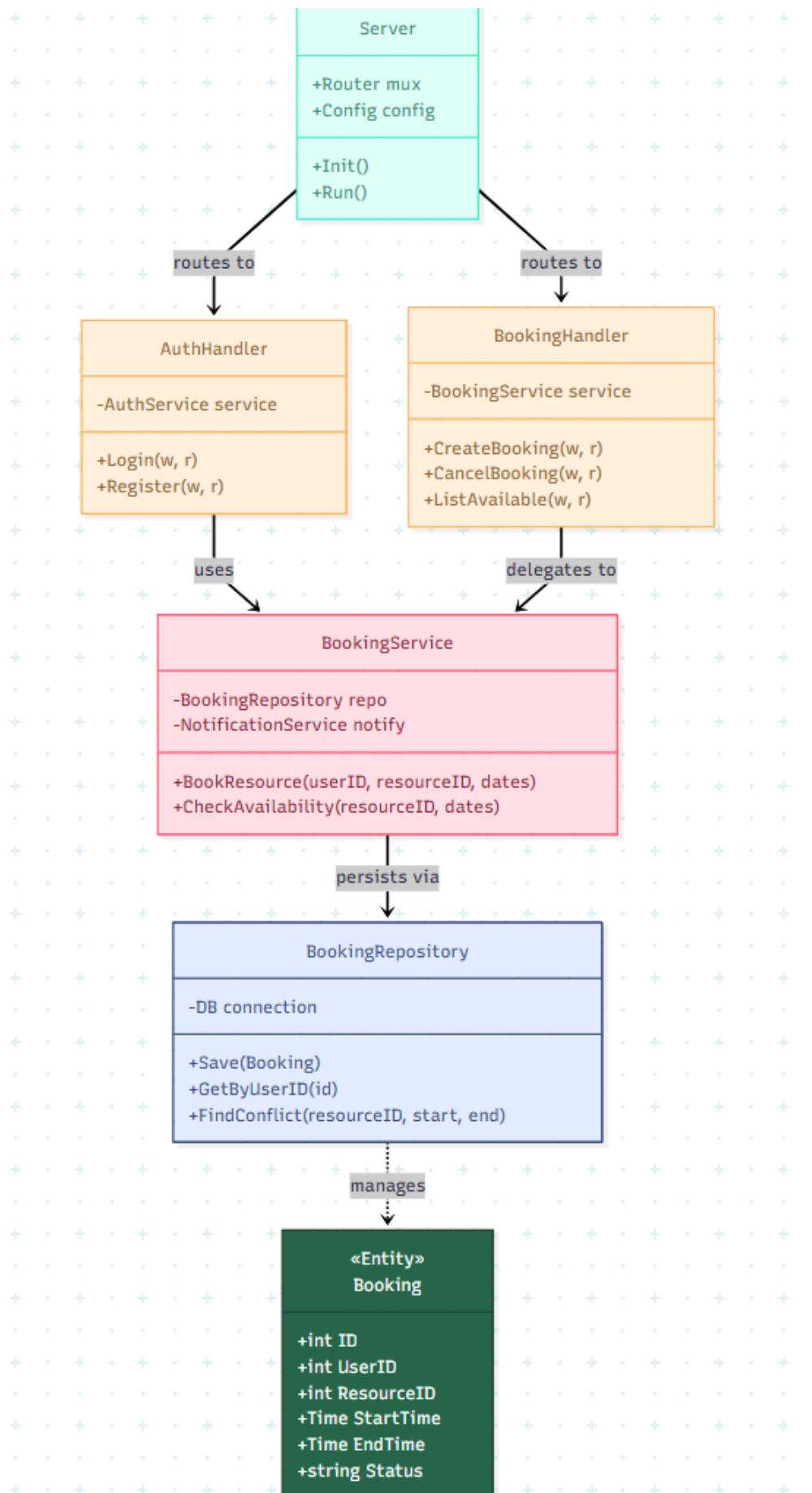
USER ||--o{ BOOKING : places

RESOURCE ||--o{ BOOKING : "is allocated to"

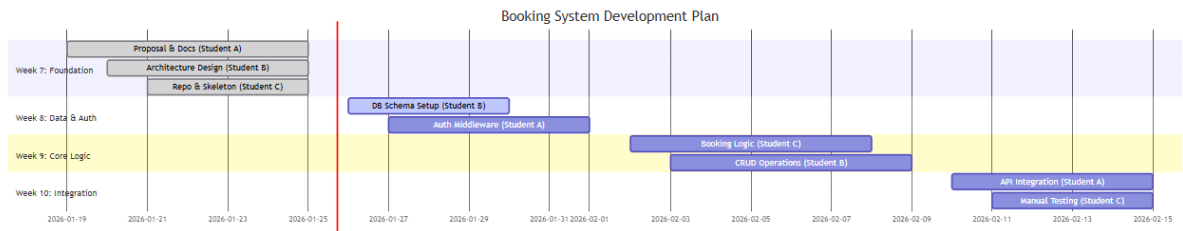
```
USER {  
    int id PK  
    string email  
    string password_hash  
    string role  
}
```

```
RESOURCE {  
    int id PK  
    string name  
    string category  
    float hourly_rate  
}
```

```
BOOKING {  
    int id PK  
    int user_id FK  
    int resource_id FK  
    datetime start_time  
    datetime end_time  
    string status  
}
```



3. Project Plan (Gantt Chart)



4. How to Run

1. Run the application:

go run .

Github link: https://github.com/Hazardiusoms/ADP_Project/tree/feature/skeleton-branch